



COVERAGE ANALYSIS

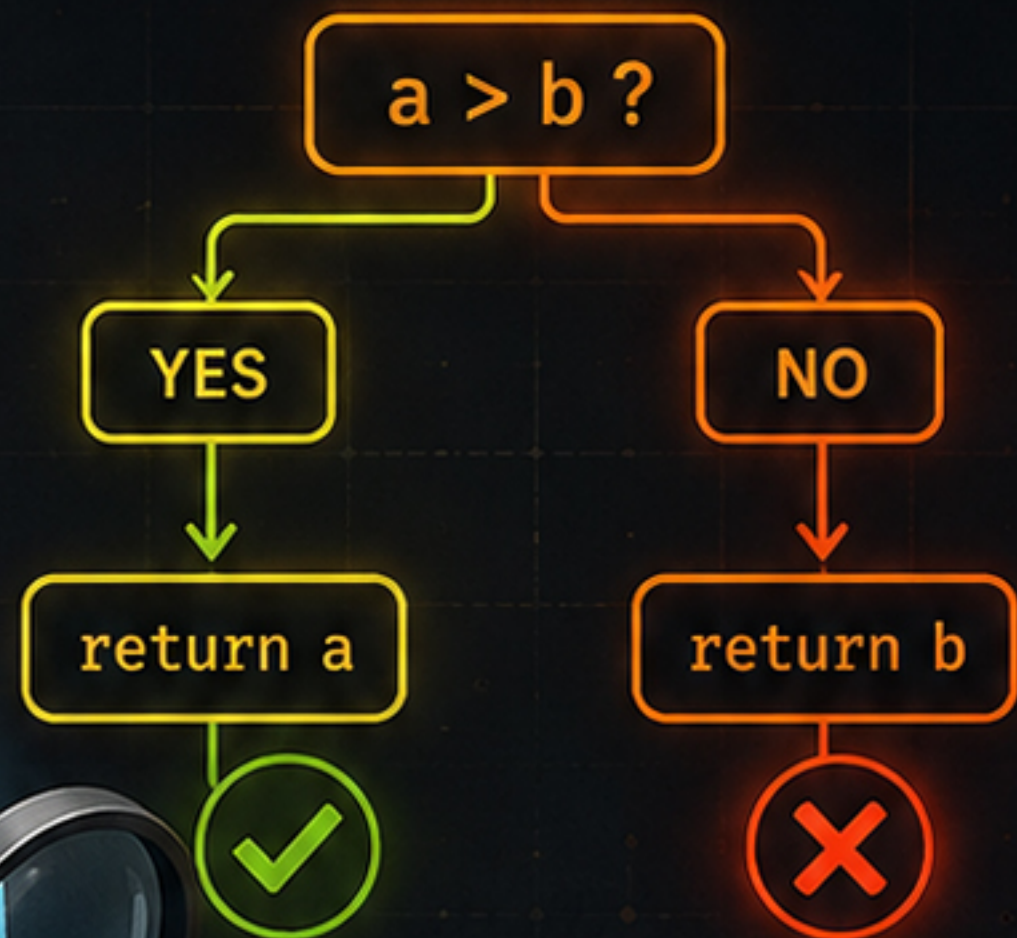


KNOW WHAT YOUR **TESTS EXECUTE**.



FIND WHAT THEY **MISS**.

```
func Max(a, b int) int {  
    if a > b {  
        return a  
    }  
  
    return b  
}
```



COVERAGE

87%



CHAPTER 4 • GO TESTING HANDBOOK

WHAT IS CODE COVERAGE?

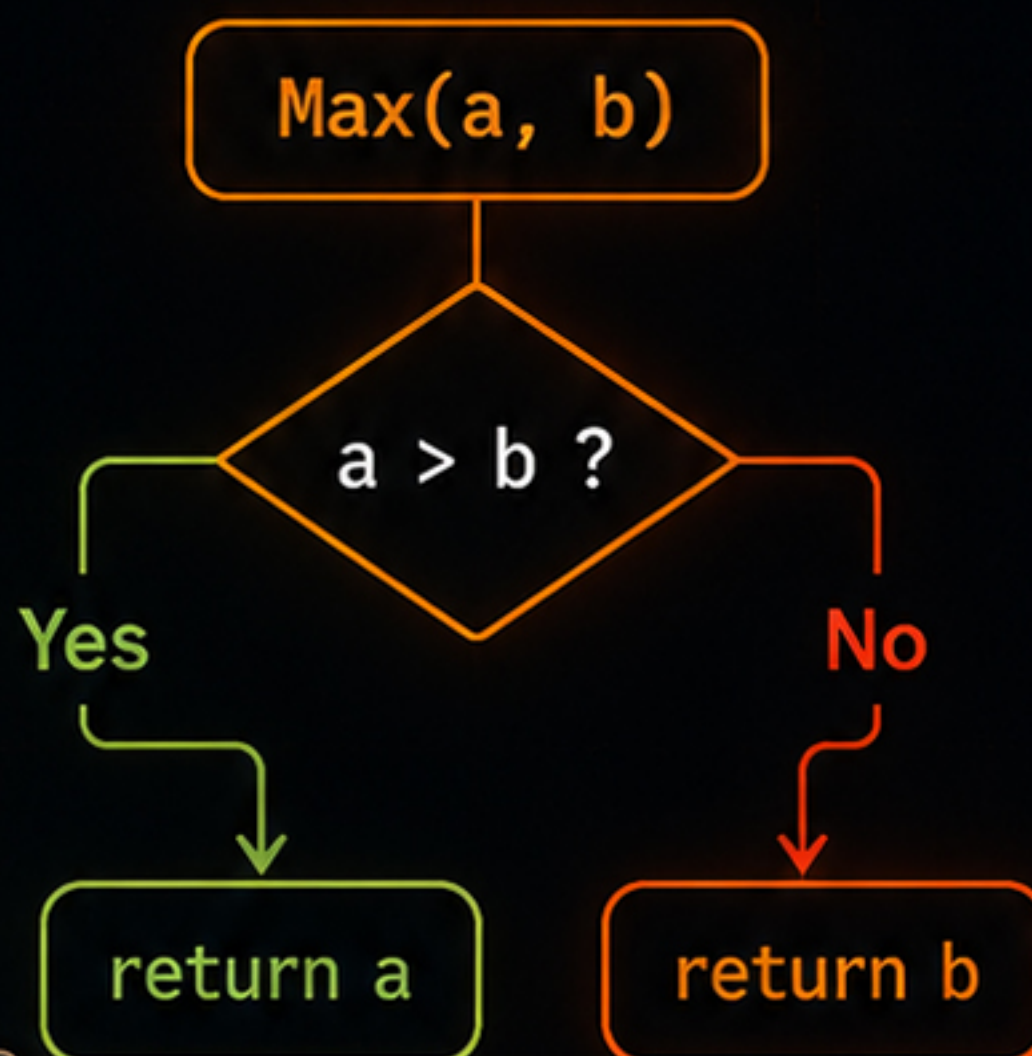


It measures **how much** of your code **is executed** while running tests.



Coverage shows **what ran**, not whether **it was correct**.

```
func Max(a, b int) int {  
    if a > b {  
        return a  
    }  
  
    return b  
}
```



COVERAGE = MEASURES EXECUTION

NOT EVERY PATH GETS TESTED



A function can have **multiple execution paths**.



If your tests cover **only one branch**, the others remain **untested**.



```
func Max(a, b int) int {  
    if a > b {  
        return a  
    }  
    return b  
}
```



Max(a, b)

Yes

a > b ?

No

return a

return b



Not covered
by tests



Covered by tests

```
// Only this path is tested  
func TestMax(t *testing.T) {  
    Max(10, 5) // a > b → true  
}
```


100% COVERAGE ≠ BUG-FREE CODE



Running code
isn't enough.



A test without
assertions can
report 100% coverage
while verifying
nothing.



```
func TestAdd(t *testing.T) {  
    Add(2, 3)  
    // No assertions!  
}
```



TEST PASSES
EVEN IF WRONG!



Code is executed.
Coverage increases.
But behavior isn't
verified.



COVERAGE TELLS YOU WHAT RAN,
NOT WHETHER IT WAS CORRECT.

MEASURE EXECUTION, NOT QUALITY



COVERAGE

tells you
what executed.

```
// Code executed  
func Add(a, b int) int {  
    return a + b  
}
```



ASSERTIONS

tell you
whether it behaved
correctly.

```
func TestAdd(t *testing.T) {  
    got := Add(2, 3)  
    if got != 5 {  
        t.Fatalf("expected 5,  
            got %d", got)  
    }  
}
```



BEHAVIOR VERIFIED



EXECUTION

What ran



YOU NEED
BOTH



QUALITY

What's correct



COVERAGE + ASSERTIONS = CONFIDENCE

GENERATE COVERAGE REPORTS



Quick coverage
summary:

```
$ go test -cover
```

PASS

coverage: 87.3% of statements
ok example/math 0.003s



87.3%
COVERAGE



Create detailed
reports with:

```
$ go test -coverprofile=coverage.out
```

PASS

coverage: 91.8% of statements
ok github.com/example/project 0.004s



coverage.out



Measure what **executed**.
Discover what's **missing**.

VISUALIZE MISSING TESTS



Generate an
interactive
HTML report:



```
$ go tool cover -html=coverage.out
```

coverage.html

(example/max.go)

1	package math	✓ EXECUTED
2		✓ EXECUTED
3	func Max(a, b int) int {	✓ EXECUTED
4	if a > b {	✓ EXECUTED
5	return a	✗ NOT EXECUTED
6	}	✗ NOT EXECUTED
7		✗ NOT EXECUTED
8	return b	✗ NOT EXECUTED
10	}	



GREEN = EXECUTED
This code ran during tests.



RED = NOT EXECUTED
This code was never executed.



**INSTANTLY SPOT
MISSING SCENARIOS.**



END



USE COVERAGE THE RIGHT WAY



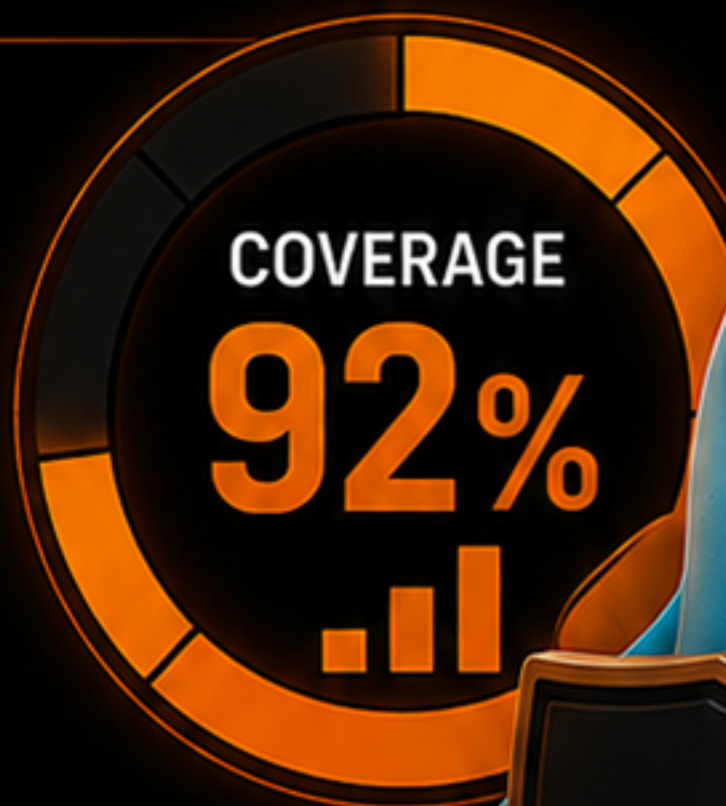
Discover
untested code



Improve
critical paths



Add
meaningful assertions



**COVERAGE IS A DIAGNOSTIC TOOL—
NOT THE GOAL.**



➔ **READ THE FULL CHAPTER
IN THE **GO TESTING HANDBOOK****



<https://github.com/golang-evangelist/.../04-coverage.md>



LIKE
if you found
this helpful



COMMENT
your thoughts
or questions



SHARE
with fellow
Gophers



SAVE
for later
reference